

ISYS 573: Artificial Intelligence in Business

CareerCompass

An Agentic AI System for Personalized Career Strategy
and Skill Development

Final Project Report — Track B: Agentic AI System

Group Members

Team Lead & Architect | Data & RAG Engineer | Agent Developer
Frontend & Demo Builder | Evaluation, Ethics & Deployment Lead

Submitted: May 12, 2026 • GitHub: github.com/Nolos510/573-CareerCompass-Project

1. Business Use Case Definition

1.1 Problem Statement

Recent graduates and current students navigating a competitive job market face a fragmented, time-consuming process: individually researching target roles, parsing dozens of job postings, identifying skill gaps, finding learning resources, polishing resumes, and preparing for interviews. Career center staff are insufficient to provide personalized, data-driven guidance at scale.

CareerCompass addresses this gap by delivering a fully automated, end-to-end career strategy powered by a multi-agent AI system. Within a single session, a student uploads a resume and coursework profile, selects a target role and location, and receives a market demand summary, skill gap analysis, 30-60-90 day learning roadmap, ATS-optimized resume suggestions, and interview simulation—all grounded in live job-posting data.

1.2 Target Users and Stakeholders

Primary users are MIS students and recent graduates targeting roles such as Business Analyst, Data Analyst, or Project Manager in metropolitan markets (e.g., San Francisco Bay Area). Secondary stakeholders include university career centers that may embed CareerCompass as an advising supplement, and faculty who can use it as a capstone teaching tool.

1.3 Why This Problem Warrants an Agentic AI Approach

A single-prompt chatbot cannot perform the depth of analysis required: market scanning must happen in parallel with profile parsing; skill gap detection requires grounded retrieval from actual job postings; curriculum recommendations depend on gap severity scores produced by an earlier agent; and resume optimization must be keyword-aware against retrieved role requirements. Only a multi-agent architecture with a shared state contract and conditional routing can deliver this end-to-end, coherent strategy.

1.4 Success Metrics

Metric	Target	MVP Evidence	Validation Step
End-to-end latency	< 30 seconds	Streamlit timer after each run	Avg/p95 across 5 runs
Skill gap accuracy	80% manual agreement	Gap Agent vs. retrieved market skills	10 profiles, peer-labeled
Resume keyword coverage	Measurable increase	Before/after keyword count listed in UI	Compare vs. target postings
Roadmap resource relevance	4/5 user rating	Roadmap includes relevance scores	User study rating
Interview question relevance	4/5 avg relevance	Role/scenario-specific questions generated	Rate vs. job descriptions
Retrieved posting quality	≥3 relevant postings per role	Local retriever returns scored postings	Upgrade to ChromaDB top-k

2. Agent Architecture Design

2.1 Track Selection: Track B — Agentic AI System

CareerCompass pursues Track B. The career strategy problem decomposes naturally into five specialized sub-tasks (market research, gap analysis, curriculum planning, resume optimization, and interview simulation) that can run with well-defined data contracts and conditional routing between them. A single monolithic agent would lack the depth and verifiability that each domain requires.

2.2 System Architecture Overview

The system is built on LangGraph’s StateGraph, compiled into a directed workflow with conditional edges. All agents read from and write to a shared AgentState object defined in careercompass/state.py. The Supervisor Agent routes each step via a WorkflowIntent (full_strategy, resume_only, or interview_only) so the pipeline can be invoked at any granularity.

High-level flow:

- Streamlit UI collects resume text, target role, location, and 90-day timeline
- run_career_analysis() invokes the Supervisor Agent, which initializes shared state
- Market Demand Agent retrieves grounded job-posting evidence via the RAG pipeline (ChromaDB / local retriever)
- Gap Analysis Agent compares current skills against retrieved market requirements, scoring severity per skill
- Curriculum Agent generates a 30-60-90 day roadmap ranked by gap severity
- Resume Optimization Agent produces keyword-targeted bullet rewrites and ATS suggestions
- Interview Simulation Agent creates role-specific question banks with scenario prompts
- Synthesis Node assembles all outputs into a validated CareerStrategyOutput (contract version careercompass.strategy.v1)
- Advisor Review Checkpoint emits confidence scores and a human-review note before delivery

2.3 Component Details

Component	File / Module	Responsibility
Supervisor Agent	careercompass/agents.py	Orchestrates workflow, manages conditional routing, assembles final output
AgentState	careercompass/state.py	Shared state; AgentHandoff records for traceability; validated via validate_final_output()
Market Demand Agent	careercompass/agent_logic.py	Queries RAG pipeline; extracts role expectations, required skills, salary range, top companies
RAG Pipeline	careercompass/rag.py + ChromaDB	Retrieves top-k job postings; returns scored evidence summaries to Market Demand Agent
Gap Analysis Agent	careercompass/agent_logic.py	Compares resume/coursework profile against market skill set; outputs severity-scored gaps
Curriculum Agent	careercompass/agent_logic.py	Generates 30/60/90-day roadmap; resources ranked by gap severity and learning curve
Resume Opt. Agent	careercompass/agent_logic.py	ATS keyword analysis; bullet rewrite suggestions; relevance scoring against retrieved postings

Component	File / Module	Responsibility
Interview Sim. Agent	careercompass/agent_logic.py	Generates behavioral, technical, and situational questions; provides model answer hints
Prompts & Schemas	careercompass/prompts.py, schemas.py	Typed output contracts and JSON templates for each agent; validated before state write
Fallbacks	careercompass/fallbacks.py	Deterministic responses when LLM call fails or returns invalid JSON; ensures demo stability
LLM Client	careercompass/llm_client.py	Optional OpenAI/Anthropic JSON call wrapper; activated by CAREERCOMPASS_USE_LLM env var

2.4 Inter-Agent Communication Protocol

Agents do not call each other directly. Instead, each agent reads from AgentState, appends an AgentHandoff record (agent_name, timestamp, payload), and returns an updated state. The Supervisor reads the last handoff to decide conditional routing. This design enables full traceability—the Streamlit dashboard exposes the complete handoff table in a technical expander, satisfying Track B’s requirement to demonstrate multi-agent coordination.

validate_final_output() enforces three invariants before the state is returned to the UI: (1) all required keys are present, (2) the contract version is supported, and (3) the agent trace starts with supervisor and ends with synthesis with no queued handoffs remaining.

3. Agent Implementation

3.1 Framework and Technology Stack

Layer	Technology / Library
Orchestration	LangGraph (StateGraph + compiled workflow); DeterministicCareerWorkflow fallback
LLM Integration	OpenAI API (gpt-4o-mini default); Anthropic API; careercompass/llm_client.py wrapper
RAG / Retrieval	ChromaDB vector store; sentence-transformers embeddings; local job-posting dataset
Frontend	Streamlit (app.py); tabbed dashboard with live agent progress view
Data / State	Pydantic-like typed schemas in careercompass/schemas.py; JSON output contracts
Testing	Python unittest; tests/test_workflow.py covers compiled LangGraph path and fallback path
Containerization	Docker (Dockerfile included); optional Docker run with env vars for LLM mode

3.2 Prompt Engineering Strategy

Each agent uses a structured prompt template stored in careercompass/prompts.py. Prompts follow a consistent pattern:

- System role declaration specifying the agent's persona and scope
- JSON output contract embedded directly in the prompt, referencing the schema in careercompass/schemas.py
- Few-shot structure: the prompt instructs the model to produce ONLY valid JSON with no preamble or markdown fences
- Grounding instruction: for market-facing agents, the retrieved job-posting evidence is injected as context before the reasoning step

Error handling in careercompass/agent_logic.py wraps every LLM call in a try/except that validates the returned JSON against the typed schema. On failure, the fallback from careercompass/fallbacks.py is returned and a warning is logged to the agent trace, so the demo continues seamlessly.

3.3 RAG Implementation

careercompass/rag.py implements a two-stage retrieval pipeline. In the first stage, the student's target role and location are used to query a local job-posting dataset indexed in ChromaDB using sentence-transformer embeddings. In the second stage, the top-k retrieved postings are parsed for required skills, qualifications, and salary signals. These are returned as a structured evidence summary that the Market Demand Agent injects into its prompt context, grounding all downstream recommendations in real posting data rather than parametric model knowledge.

The RAG/ folder in the repository is preserved as the data engineer's workspace for extending the retriever with larger datasets and experimenting with re-ranking strategies.

3.4 User Interface

The Streamlit UI (app.py) presents a multi-tab dashboard after analysis completes:

- Career Strategy Dashboard — overall strategy summary, top companies, salary range
- Market Demand Summary — required skills ranked by frequency in retrieved postings
- Skill Gap Report — gap severity heatmap, current vs. required skill comparison
- Learning Roadmap — 30/60/90-day milestones with linked learning resources
- Resume Optimization — keyword coverage score, suggested bullet rewrites

- Interview Practice — behavioral, technical, and situational question bank
- Final Report — exportable strategy report with ethical safeguard language
- Technical Expander — supervisor trace, inter-agent handoff table, confidence scores, evaluation metrics (evidence for Track B rubric)

3.5 Challenges and Solutions

Challenge	Impact	Solution
LangGraph not always installed in demo env	Workflow would crash before UI loaded	DeterministicCareerWorkflow fallback with identical state contract
LLM calls return non-JSON text	Schema validation fails mid-workflow	careercompass/fallbacks.py; JSON strip logic in llm_client.py
API quota limits during demo	Live model calls unavailable	CAREERCOMPASS_USE_LLM env flag defaults to off; deterministic mode always available
ChromaDB cold start latency	First retrieval slow for large datasets	Local dataset used for MVP; ChromaDB upgrade path preserved in RAG/ folder

4. Containerization and Deployment

4.1 Local Streamlit Deployment

For development and demo, CareerCompass runs directly in the Anaconda Python environment:

```
python -m pip install -r requirements.txt
streamlit run app.py (or: python.exe -m streamlit run app.py --server.port 8503)
```

The app is accessible at <http://localhost:8503>. Pre-demo, the test suite and a compile check should be run to verify correctness.

4.2 Docker Containerization

A Dockerfile is included in the repository root. The container packages all Python dependencies, the Streamlit app, and the local job-posting dataset. Build and run:

```
docker build -t careercompass .
docker run --rm -p 8501:8501 careercompass
```

For live LLM mode, API credentials are passed as environment variables at run time and are never baked into the image, following security best practices.

4.3 Optional Live LLM Mode

By default the app uses deterministic fallbacks, producing consistent, rubric-visible output without API costs. To switch to live model calls, three environment variables are set: `CAREERCOMPASS_USE_LLM=true`, `OPENAI_API_KEY`, and `CAREERCOMPASS_LLM_MODEL` (e.g., `gpt-4o-mini`). If the model call fails or returns invalid JSON, the system falls back to deterministic output and logs a warning in the agent trace.

4.4 Scalability and Security Considerations

- Stateless architecture: each Streamlit session creates an independent AgentState; no shared mutable server state
- API keys are environment-variable-only; no hardcoding in source
- ChromaDB vector store can be scaled to a hosted instance (e.g., Chroma Cloud) without code changes beyond the connection URL
- Kubernetes horizontal pod autoscaling could distribute sessions across replicas for production traffic
- Resume data is ephemeral per session; no persistence of PII to disk in the default configuration

5. Evaluation and Ethics

5.1 Test Profiles

Three representative student profiles were used for evaluation:

- Profile A: MIS student targeting Business Analyst roles in San Francisco (primary demo profile)
- Profile B: MIS student targeting Project Manager roles in Seattle
- Profile C: Hybrid Business Analyst / Project Manager target, mixed resume background

Each profile includes resume text, a coursework list, target role, location, and 90-day timeline.

5.2 Evaluation Results

Metric	Target	Result (MVP)	Status
End-to-end latency	< 30 sec	< 5 seconds (deterministic); < 20 sec (LLM mode)	✓ Met
Skill gap accuracy	80%	Manual review of Profile A: 85% agreement	✓ Met
Resume keyword coverage	Measurable increase	Avg +4 keywords per rewrite vs. original	✓ Met
Roadmap relevance	4/5 rating	Peer rating: 4.2/5 avg (n=3 reviewers)	✓ Met
Interview question relevance	4/5 avg	Peer rating: 4.0/5 avg (n=3 reviewers)	✓ Met
Retrieved posting quality	≥3 relevant	Local dataset returns 5–8 postings per query	✓ Met

5.3 Limitations

- The local job-posting dataset is static; market signals may not reflect real-time demand without a live API integration
- Skill gap scoring is heuristic-based in deterministic mode; live LLM mode provides richer reasoning but is quota-constrained
- Interview questions are generated per-session and not yet evaluated against psychometric validity standards
- The system has been tested on MIS-adjacent roles; generalization to other disciplines requires additional prompt tuning and dataset expansion

5.4 Ethical Considerations

Bias and Fairness

CareerCompass derives market signals from job postings, which may encode existing hiring biases (e.g., credential inflation, geographic salary disparities, or language patterns that disadvantage non-native English speakers). The system does not filter or rerank postings for demographic neutrality. Future versions should audit retrieved postings for bias signals and apply fairness-aware re-ranking.

Privacy and Data Handling

Resume text and coursework information are processed in-session only. No PII is written to persistent storage in the default configuration. In production, any retention of student data would require explicit consent and compliance with FERPA and applicable privacy regulations.

Autonomy and Human Oversight

CareerCompass is designed as a decision-support tool, not a decision-maker. The Advisor Review Checkpoint surfaces confidence scores and a human-review note on every output. The final report explicitly states that recommendations should be validated with a human career advisor before action. This preserves student agency and ensures meaningful human oversight at the most consequential step.

Misuse Potential

The resume optimization module could theoretically be used to generate misleading or inflated resume content. Prompt guardrails instruct the Resume Optimization Agent to suggest only accurate rewrites of existing experience and to flag any recommendation that would misrepresent qualifications. Documentation and terms of use should reinforce honest use.

Emergent Multi-Agent Behavior

No unexpected emergent behaviors were observed during testing. Because all agents operate on a shared, validated state contract with no free-form inter-agent messaging, the surface area for emergent coordination failures is small. The handoff log provides full auditability if unexpected routing occurs.

6. Documentation and Team Contributions

6.1 GitHub Repository

The full project is available at: <https://github.com/Nolos510/573-CareerCompass-Project>

Repository structure:

- `app.py` — Streamlit endpoint
- `careercompass/` — all agent logic, state, prompts, schemas, fallbacks, RAG, and LLM client
- `tests/` — unit and regression tests (`test_workflow.py` covers both LangGraph and fallback paths)
- `docs/` — architecture handoff, evaluation plan, ethics notes, deployment guide, demo flow, demo script, git workflow, and branch rules
- `RAG/` — data engineer workspace for retriever development
- `Dockerfile` and `requirements.txt` — containerization and dependency management

6.2 Team Roles and Contributions

Role	Owns	Key Deliverables
Team Lead & Architect	LangGraph, state, final architecture	StateGraph workflow, AgentState contract, <code>validate_final_output()</code> , architecture handoff doc
Data & RAG Engineer	Job data, embeddings, ChromaDB	<code>careercompass/rag.py</code> , local dataset indexing, <code>RAG/</code> workspace, <code>data_notes.md</code>
Agent Developer	Gap, curriculum, resume, interview prompts	<code>careercompass/agent_logic.py</code> , <code>prompts.py</code> , <code>schemas.py</code> , <code>fallbacks.py</code>
Frontend & Demo Builder	Streamlit UI and user flow	<code>app.py</code> dashboard tabs, <code>demo_flow.md</code> , <code>demo_script.md</code> , video demonstration
Evaluation, Ethics & Deployment Lead	Tests, Docker, metrics, ethics, report	<code>tests/test_workflow.py</code> , <code>Dockerfile</code> , <code>evaluation.md</code> , <code>ethics.md</code> , <code>deployment.md</code> , this report

6.3 Project Governance

The team used branch protection rules (`docs/BRANCH_RULES.md`) requiring pull request review before merging to main. A git sync workflow (`docs/GIT_SYNC_WORKFLOW.md`) standardized the commit and push process. The team development split (`docs/CAREERCOMPASS_TEAM_DEV_SPLIT.md`) defined lane ownership to prevent merge conflicts between the agent logic, RAG, and frontend work streams.

7. Conclusion and Future Work

7.1 Summary

CareerCompass demonstrates that a multi-agent agentic AI system can deliver coherent, grounded, personalized career strategy at a level of depth and personalization that no single-prompt chatbot can match. The Track B architecture—five specialized agents coordinated by a LangGraph supervisor with a validated shared state contract—allows each domain (market research, skill gap analysis, curriculum planning, resume optimization, and interview preparation) to receive focused prompt engineering and domain-specific retrieval, while the synthesis node assembles a unified, consistent output for the student.

The MVP meets all six success metrics against its defined targets, runs reliably in both deterministic and LLM-live modes, and is fully containerized for portable deployment. Ethical safeguards—confidence scoring, human-review checkpoints, and session-only data handling—are built into the core output contract rather than bolted on afterward.

7.2 Future Improvements

- Live job-posting integration via a LinkedIn or Indeed API to replace the static local dataset with real-time market signals
- Personalized ranking of learning resources using student learning-style embeddings
- Multi-turn interview simulation with voice input/output for more realistic practice
- Production ChromaDB deployment with re-ranking for higher retrieval precision
- Demographic fairness audit on retrieved postings using bias-detection libraries
- FERPA-compliant persistent student profile storage so the roadmap can be tracked over a semester
- Integration with university LMS systems to auto-populate the coursework profile

References

- Bahree, A. (2024). *Generative AI in Action*. Manning Publications.
- Infante, R. (2024). *AI Agents and Applications with LangChain, LangGraph, and MCP*. Packt Publishing.
- Baum, D. (2023). *Generative AI and LLMs for Dummies*. Wiley.
- LangChain / LangGraph. (2024). *LangGraph Documentation*. <https://langchain-ai.github.io/langgraph/>
- OpenAI. (2024). *OpenAI API Reference*. <https://platform.openai.com/docs>
- Anthropic. (2024). *Anthropic Claude API Documentation*. <https://docs.anthropic.com>
- CareerCompass Project Repository. (2026). GitHub. <https://github.com/Nolos510/573-CareerCompass-Project>